Git 版控系统

Git Version Control System

November 7, 2026

WHUAI Team

Ostrichbeta Chan

1. 开始之前

2. 保存与检查版本

3. branch 与合并

4. 分享与协作

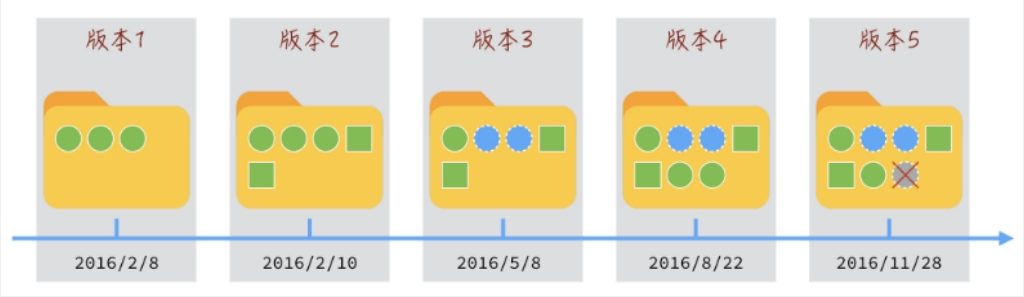
今天要完成的三件事

1. 保存、检查并修正专案的版本。
2. 在 branch 上开发，并解决一次合并冲突。
3. 把 branch 推送到 GitHub，完成一次 Pull Request。

今天的方式

透过同一个小专案边做边学，每完成一段就动手练习。

我们想解决什么问题？



- 哪个版本可以正常执行？
- 这次修改了什么？为什么？
- 两个人的修改怎么合在一起？

Git 保存版本；GitHub 协助分享与审阅

Git：电脑上的版本控制工具

- 保存 commit 与查看历史。
- 建立 branch、合并修改。
- 本机操作不需要网路。

GitHub：托管 Git 仓库的平台

- 以远端仓库分享专案。
- 审阅程式码、讨论修改。
- 用 Pull Request 提出合并请求。

执行 commit 只会建立本机 commit；需 push 才会上传。

先确认工具与 commit 作者资讯

```
git --version
git config --global user.name "Alex Chen"
git config --global user.email "alex@example.com"
```

- Git 的 user.name、user.email 是 commit 纪录的作者资讯，不是 GitHub 登入资讯。
- Windows 预设没有 Git；安装 Git for Windows，使用 Git Bash。macOS、Linux 用终端机。
- 使用 Git 2.28 以上版本；建议事先确认可连线至 GitHub，并准备 GitHub 帐号。

指令约定

只输入指令，不输入提示符号；YOUR-USER 要换成自己的帐号。
可以向 AI 求助，但课堂上请亲手输入每一条命令。

1. 开始之前

2. 保存与检查版本

3. branch 与合并

4. 分享与协作

今天只用这个小专案

```
def average(scores):
    return sum(scores) / len(scores)

print("Average:", average([70, 80, 90]))
```

- grades.py：计算平均分数。
- README.md：说明输入与输出。
- 把讲义中的起始档复制到**独立的练习目录**。

今天只用这个小专案

```
def average(scores):
    return sum(scores) / len(scores)

print("Average:", average([70, 80, 90]))
```

- grades.py：计算平均分数。
- README.md：说明输入与输出。
- 把讲义中的起始档复制到**独立的练习目录**。

重点不是范例程式怎么运作，而是从档案内容的变化学 Git。

一次 commit，经过三个地方



- 工作目录：目前看到、正在修改的档案。
- 暂存区：准备纳入下一次 commit 的档案内容。
- commit：把暂存区的专案状态保存成一个版本。

让练习目录开始保存版本

```
pwd
git init -b main
git status
```

- 先确认目前在自己的 `git-grade-lab` 目录。
- `git init -b main` 会在建立 repository 时把初始 branch 设为 `main`。
- 旧版 Git 或未指定初始 branch 的 `git init` 可能建立 `master`。
- **`.git` 保存 repository 的历史与设定，请勿删除。**

遇到问题，先读 git status

读懂三种常见状态

`Untracked`	Git 还没开始追踪这个新档案，尚未纳入版本控制。
`Not staged`	已追踪的档案有修改，尚未加入暂存区。
`To be committed`	变更已进暂存区，下次 commit 会包含这些内容。

- 同一个档案可以同时有已 staged 和尚未 staged 的变更。
- `working tree clean`：本机没有待 commit 的变更，不代表已 push。

遇到问题，先读 git status

下面是 `git status` 输出的重点节录，省略其他提示。

Untracked

```
Untracked files:
    README.md
    grades.py
```

Not staged

```
Changes not staged for commit:
    modified:   README.md
```

遇到问题，先读 git status

To be committed

`git add README.md` 后，修改已暂存，等待 commit。

```
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   README.md
```

保存第一个版本

```
git add README.md grades.py
git status
git commit -m "Add grade calculator"
git status
```

- `add`：选择这次要保存的内容。
- `commit -m`：保存版本，说明这次改了什么。
- commit message 格式没有强制规定，但内容要精确、简洁。

先 add，再修改：哪一版会被 commit？

1. 在 README 加入 `Input: a list of scores.`
2. 执行 `git add README.md`。
3. 再加入 `Output: the average score.`，先不要 add。

```
git status
git diff
git diff --staged
```

- `diff`：工作目录与暂存区的差异。
- `diff --staged`：暂存区与上次 commit 的差异。

练习 1：把两次修改分开保存

1. 承接上一页，只 commit Input 那一行。
2. 确认 Output 那一行还留在工作目录。
3. 将 Output 那一行留到下一个 commit。

完成条件

- 至少三个 commit：初始专案、输入说明、输出说明。
- 工作目录干净，并能说明两次 diff 各自比较的对象。

有些档案不应该进入版本纪录

在专案根目录建立 `.gitignore`：

```
.env
id_rsa
__pycache__/
build/
```

本机设定与凭证

- `.env`
- `id_rsa`

自动生成档案

- `__pycache__/`
- `build/`

规则对尚未追踪的档案生效；已追踪的档案不会自动取消追踪。

一起暂存目前目录的变更

. 表示目前目录；这个操作也包含子目录。

```
git status
git add .
git diff --staged
```

- 新增、修改与删除都会暂存；符合忽略规则的未追踪档案会略过。
- commit 前先读 `git diff --staged`，确认要保存的变更。
- 只想选部分档案时，仍可用 `git add` 档名。

用历史回答：改了什么？为什么？

```
git log --oneline
git show HEAD
```

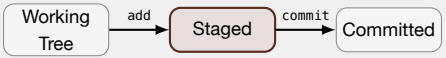
- `--oneline` 让每个 commit 只占一行，显示缩短的 commit ID 与 message 首行。
- HEAD 通常经由目前 branch 指向该 branch 的最新 commit。
- `show HEAD` 可以检查最近一次 commit 的内容。
- 出现分页画面时按 q 返回终端机。

修正错误前，先判断问题在哪个阶段

图中亮色节点标出操作前相关变更的位置。

已 staged、未 commit：取消暂存并保留工作区档案

```
git restore --staged README.md
```



Working Tree 未 staged 的修改：确定不要才丢弃

```
git restore README.md
```



已 commit 的变更：新增 revert commit 抵销，保留历史

```
git revert --no-edit HEAD
```



练习 2：撤销错误 commit，保留历史

1. 把 README 第一行改成 `# Broken calculator`。
2. 保存为一个 commit，讯息为 `Change heading by mistake`。
3. 先检查历史，再用 `git revert --no-edit HEAD` 抵销这次 commit。

完成条件

标题恢复、工作目录干净；历史中同时保留错误 commit 与抵销 commit。

revert 抵销变更，并不删除原始 commit。

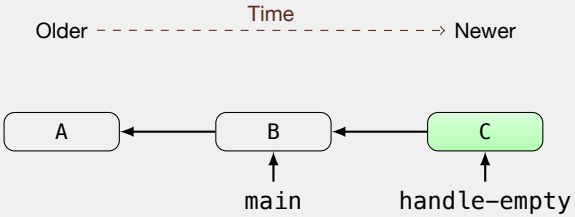
1. 开始之前

2. 保存与检查版本

3. branch 与合并

4. 分享与协作

branch 代表另一条开发路线



Commit arrows point to parents

- branch 名称指向某个 commit；产生新 commit 时，目前 branch 会往前移动。
- 图中功能 branch 多出 commit C，`main` 仍停在 B。
- HEAD 通常指向目前所在的 branch。

在功能 branch 处理空的分数清单

```
git switch -c handle-empty
```

把 average 函式改成：

```
def average(scores):
    if not scores:
        return 0
    return sum(scores) / len(scores)
```

```
git add grades.py
git commit -m "Handle empty score lists"
```

把功能合并回 main

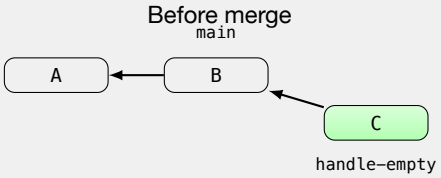
```
git switch main
git merge handle-empty
git log --oneline --graph --all
```

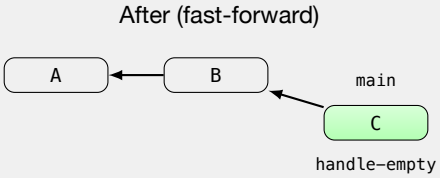
- `merge handle-empty` 把功能并入目前所在的 branch。
- 这次 `main` 没有额外 commit，可以直接快转（fast-forward）。
- 合并后功能 branch 名称仍保留，不会自动删除。

把功能合并回 main

```
git switch main
git merge handle-empty
git log --oneline --graph --all
```

- `merge handle-empty` 把功能并入目前所在的 branch。
- 这次 `main` 没有额外 commit，可以直接快转（fast-forward）。
- 合并后功能 branch 名称仍保留，不会自动删除。





练习 3：确认功能已合并回 main

1. 完成上一段的 branch 建立、修改、commit 与合并。
2. 用 `git branch` 确认目前在 `main`。
3. 开启 `grades.py`，确认空清单的判断已出现在 `main`。

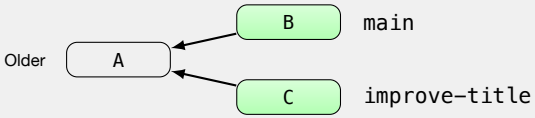
完成条件

`main` 已有新功能、工作目录干净；能说明为何要先切换 branch 再合并。

可选检查

有 Python 时，测试一般清单与空清单。

两边都修改同一个标题，怎么办？



`improve-title`

README 第一行改为：

```
# Team grade calculator
```

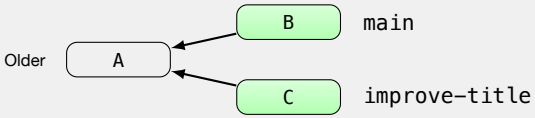
`main`

README 第一行改为：

```
# Class grade calculator
```

回到 `main`，执行 `git merge improve-title`。

两边都修改同一个标题，怎么办？



`improve-title`

README 第一行改为：

```
# Team grade calculator
```

`main`

README 第一行改为：

```
# Class grade calculator
```

回到 `main`，执行 `git merge improve-title`。

Git 无法判断该保留哪个标题，需要我们自行决定。

读懂冲突标记，保留双方的用意

开启发生冲突的 README.md，查看 Git 标出的冲突内容。

```
<<<<<<< HEAD
# Class grade calculator
=======
# Team grade calculator
>>>>>>> improve-title
```

- 上半部是目前 main 的内容，下半部是要并入的 branch 内容。
- 第一行改为 `# Team grade calculator for class`。
- 删除全部冲突标记，保留其余说明。

解决后，还要标记并完成 commit

```
git status
git add README.md
git commit -m "Combine team and class descriptions"
git status
```

- 编辑档案并检查结果。
- add 告诉 Git：这个档案的冲突已处理。
- commit 完成合并；之后再检查与验证。
- 取消尚未完成的合并：`git merge --abort`。

练习 4：完成一次有冲突的合并

1. 依讲义建立两个修改标题的 commit，重现冲突。
2. 整合双方意思，暂存并完成 merge commit。
3. 用 `git log --oneline --graph --all` 检查历史。

完成条件

- `main` 的标题保留 Team 与 class，其余说明也要保留。
- 没有冲突标记，工作目录干净。
- 能解释编辑后为何还要 add 与 commit。

1. 开始之前

2. 保存与检查版本

3. branch 与合并

4. 分享与协作

远端是另一份仓库；origin 是它的别名

`main`	自己电脑上的 branch。
`origin`	本机给远端网址取的别名。
`origin/main`	本机记录的远端 `main` 状态，可能尚未更新。

- `push`：把本机 commit 送到远端。
- `fetch`：取得远端 commit，更新本机记录的远端状态。
- fetch 不会直接更新目前工作目录的档案。

把自己的练习仓库推到 GitHub

先在 GitHub 建立 `git-grade-lab` 空仓库，先不新增 README、授权或.gitignore。



- 本机已有 repository，请看 GitHub 空仓库页的 push an existing repository 区块。
- 图片是范例，请复制自己 repository 的 URL，不要照抄图片中的帐号。

```
git remote add origin https://github.com/YOUR-USER/git-grade-lab.git
git push -u origin main
```

课堂已使用 `main`，GitHub 指示的 `git branch -M main` 不必重做。

取得既有专案，使用 clone

```
git clone https://github.com/YOUR-USER/git-grade-lab.git git-grade-copy
cd git-grade-copy
git status
```

- clone 会取得仓库，并设定 origin 与预设 branch 的上游。
- 复制好之后不需要再 init。
- 回到**原本练习目录**继续下一段。

只有远端有新增 commit、自己没有额外 commit 时，可以：

```
git pull --ff-only
```

push 被拒绝：先整合远端更新

情境：在 `main`，两边各有对方没有的 commit，工作目录干净。

```
git fetch origin
git merge --no-edit origin/main
git push
```

- 没有内容冲突时，Git 可以直接整合。
- 若有冲突，依序编辑、add、commit，再 push。
- `--no-edit` 接受自动产生的合并讯息。

两边各有新 commit，不一定代表内容冲突。

用 Pull Request 让同学审阅修改

```
git switch -c docs-usage
```

在 README 加上使用说明后：

```
git add README.md
git commit -m "Explain how to run the calculator"
git push -u origin docs-usage
```

- 在 GitHub 开 PR：base 选 `main`，compare 选 `docs-usage`。
- 说明目的、内容与检查方式，请同学审阅。
- 修改后继续 commit 并 push 到同一 branch，原本的 PR 就会更新。

练习 5：两人完成一次审阅与合并

1. 各自在自己的仓库提出 `docs-usage` 的 PR。
2. 交换连结，检查用法是否清楚，留下具体建议。
3. 作者回应并更新同一个 PR，再由有权限的人合并。
4. 回到本机 `main`，取得合并后的版本。

```
git switch main
git pull --ff-only
```

完成条件

PR 有说明与审阅纪录；本机 `main` 有使用说明，工作目录干净。

课后想一想

1. add 后又改档案，下次 commit 会保存哪一版？
2. commit 成功后，GitHub 就一定看得到吗？
3. 把 feature 合进 `main`，应该先切到哪个 branch？
4. 编辑完冲突档案，还需要哪两个步骤？
5. PR 收到意见后，要怎么更新同一个 PR？

日常工作：先看状态，再选下一步

保存与检查

- `git status`
- `git diff`
- `git diff --staged`
- `git add 档名`
- `git commit -m " 说明"`
- `git log --oneline`

branch 与协作

- `git switch -c branch 名称`
- `git switch main`
- `git merge branch 名称`
- `git push -u origin branch 名称`
- `git pull --ff-only`

5. 附录

课前准备：工具、编辑器与认证

- 安装 Git，选好编辑器，确认自己知道如何保存档案。
- 若使用 VS Code 且已安装 `code` 命令，可设定：

```
git config --global core.editor "code --wait"
```

- HTTPS 认证：Git Credential Manager 或 GitHub CLI。
- 用个人的测试仓库确认可以 push；详细流程见讲义。
- 若已有其他可用的编辑器与认证方式，可以继续使用。

Token 不要放在程式码、远端网址或截图中。

先把基本流程练熟，再学进阶操作

- `stash`：暂时收起尚未 commit 的工作。
- `rebase / cherry-pick`：重整或搬移 commit。
- `reset / reflog`：移动 branch、查找本机操作纪录。

恢复能力有边界

- Git 没有保存过的内容，不能保证找回。
- hard reset 会覆盖工作目录内容，本次练习不需要用到它。
- 改写已分享的历史前，先与团队确认工作方式。

课后练习与官方参考

- Pro Git：繁体中文版本
- Git diff：比较未暂存与已暂存的内容
- Git restore、Git revert、gitignore
- GitHub：设定 Git 认证
- GitHub：Pull Request 与程式码审阅

下次拿自己的小专案再做一次

建立 branch，完成一项小修改，检查 diff，commit 并请同学审阅。